

State Space Models :
From Kalman to HiPPO to S4 to Mamba

Roadmap

- 1 State space models and Kalman filter
- 2 From recurrence to convolution and FFT
- 3 HiPPO slowly and carefully
- 4 S4 step by step
- 5 Mamba step by step

The basic state space model

A state space model separates

- a hidden state x_t (internal memory), and
- an observed output y_t .

Discrete-time form:

$$x_{t+1} = Ax_t + Bu_t, \quad y_t = Cx_t + Du_t.$$

Continuous-time form:

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t).$$

Interpretation:

- A = memory dynamics,
- B = how input is written into memory,
- C = how memory is read out,
- D = direct skip path.

Kalman filter: formulas only

For a linear-Gaussian system,

$$x_t = Ax_{t-1} + w_t, \quad y_t = Cx_t + v_t,$$

with

$$w_t \sim \mathcal{N}(0, Q), \quad v_t \sim \mathcal{N}(0, R),$$

the Kalman filter recursion is

$$\begin{aligned}\hat{x}_{t|t-1} &= A\hat{x}_{t-1|t-1}, \\ P_{t|t-1} &= AP_{t-1|t-1}A^\top + Q, \\ K_t &= P_{t|t-1}C^\top (CP_{t|t-1}C^\top + R)^{-1}, \\ \hat{x}_{t|t} &= \hat{x}_{t|t-1} + K_t(y_t - C\hat{x}_{t|t-1}), \\ P_{t|t} &= (I - K_tC)P_{t|t-1}.\end{aligned}$$

Today we will *not* derive these. We only keep the main idea:

a hidden state compresses the past, and a structured update controls memory.

Why Kalman belongs in this story

What survives from Kalman filtering into modern SSM layers?

- ① The state x_t is the summary of everything important from the past.
- ② The matrix A controls forgetting, persistence, and oscillation.
- ③ The matrix C turns internal memory back into an output.

Then modern deep-learning SSMs ask two new questions:

- How do we choose A so the state remembers *very long* histories?
- How do we compute the resulting layer *fast* on long sequences?

Those two questions lead to HiPPO, S4, and then Mamba.

The deterministic LTI recurrence used by S4-style layers

Strip away the explicit noise and consider the linear time-invariant recurrence

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k, \quad y_k = Cx_k.$$

Assume $x_0 = 0$ for simplicity. The central fact is:

Key theorem

Any discrete LTI state space model is a convolution operator on the input sequence.

Everything in S4 depends on exploiting this theorem *computationally*.

Step 1: unroll the recurrence

Start from

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k.$$

Repeated substitution gives

$$\begin{aligned}x_k &= \bar{A}(\bar{A}x_{k-2} + \bar{B}u_{k-1}) + \bar{B}u_k \\&= \bar{A}^2x_{k-2} + \bar{A}\bar{B}u_{k-1} + \bar{B}u_k \\&= \bar{A}^kx_0 + \sum_{j=1}^k \bar{A}^{k-j}\bar{B}u_j.\end{aligned}$$

With $x_0 = 0$,

$$x_k = \sum_{j=1}^k \bar{A}^{k-j}\bar{B}u_j.$$

Applying C ,

$$y_k = \sum_{j=1}^k C\bar{A}^{k-j}\bar{B}u_j.$$

Step 2: identify the convolution kernel

Define the impulse-response coefficients

$$K_\ell := C\bar{A}^\ell\bar{B}, \quad \ell \geq 0.$$

Then

$$y_k = \sum_{j=1}^k K_{k-j} u_j = (K * u)_k.$$

So the SSM is exactly a long convolution. Equivalent transfer-function view:

$$Y(z) = H(z)U(z), \quad H(z) = C(I - \bar{A}z^{-1})^{-1}\bar{B}.$$

The recurrence view and the convolution view are the same operator.

Why this matters computationally

If we run the recurrence directly,

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \cdots \rightarrow x_L,$$

then time step k depends on time step $k - 1$, so the computation is sequential. But once we rewrite the layer as

$$y = K * u,$$

we can use convolution algorithms instead of a time loop. This gives two advantages:

- ➊ **Parallelism across positions:** all output positions can be computed together.
- ➋ **FFT acceleration:** Toeplitz convolution can be turned into pointwise multiplication in the frequency domain.

The FFT algorithm for the SSM convolution

Suppose we want

$$y = K * u$$

for length- L sequences. **Algorithm:**

- 1 Zero-pad K and u to a common length $M \geq 2L - 1$.
- 2 Compute the discrete Fourier transforms

$$\hat{K} = \text{FFT}(K), \quad \hat{u} = \text{FFT}(u).$$

- 3 Multiply pointwise

$$\hat{y} = \hat{K} \odot \hat{u}.$$

- 4 Invert

$$y = \text{IFFT}(\hat{y}).$$

- 5 Keep the first L output positions.

This is the standard convolution theorem:

$$\mathcal{F}(K * u) = \mathcal{F}(K) \odot \mathcal{F}(u).$$

Fourier details: DFT, padding, and linear vs. circular convolution

For a length- M vector $a = (a_0, \dots, a_{M-1})$, the discrete Fourier transform is

$$\hat{a}_r = \sum_{n=0}^{M-1} a_n \omega_M^{rn}, \quad \omega_M = e^{-2\pi i/M}, \quad r = 0, \dots, M-1.$$

The inverse transform is

$$a_n = \frac{1}{M} \sum_{r=0}^{M-1} \hat{a}_r \omega_M^{-rn}.$$

Why do we zero-pad? The DFT naturally computes *circular* convolution modulo M . To recover the ordinary *linear* convolution of two length- L sequences, choose

$$M \geq 2L - 1,$$

pad both sequences with zeros to length M , and then use the DFT. With that padding,

$$\text{IFFT}(\text{FFT}(K^{\text{pad}}) \odot \text{FFT}(u^{\text{pad}}))$$

equals the linear convolution, with no wrap-around corruption in the first $2L - 1$ entries.

Fourier details: exact step-by-step convolution algorithm

Suppose the sequence length is L and the SSM kernel is $K = (K_0, \dots, K_{L-1})$. To compute $y = K * u$ on the full sequence:

- 1 Choose $M \geq 2L - 1$ (typically a convenient FFT length).
- 2 Form padded vectors

$$K^{\text{pad}} = (K_0, \dots, K_{L-1}, 0, \dots, 0), \quad u^{\text{pad}} = (u_0, \dots, u_{L-1}, 0, \dots, 0).$$

- 3 Compute all Fourier coefficients

$$\hat{K}_r = \sum_{n=0}^{M-1} K_n^{\text{pad}} \omega_M^{rn}, \quad \hat{u}_r = \sum_{n=0}^{M-1} u_n^{\text{pad}} \omega_M^{rn}.$$

- 4 Multiply for every frequency r :

$$\hat{y}_r = \hat{K}_r \hat{u}_r.$$

- 5 Apply the inverse FFT and keep the first L entries.

The key point is that *each frequency index r is independent*. Once the transforms are computed, the multiply step is embarrassingly parallel.

Fourier details: why butterfly FFT is parallelizable

For $M = 2m$, split even and odd indices:

$$a_j^{\text{even}} = a_{2j}, \quad a_j^{\text{odd}} = a_{2j+1}.$$

Then the Cooley–Tukey decomposition says

$$\begin{aligned} \hat{a}_r &= \hat{a}_r^{\text{even}} + \omega_M^r \hat{a}_r^{\text{odd}}, \\ \hat{a}_{r+m} &= \hat{a}_r^{\text{even}} - \omega_M^r \hat{a}_r^{\text{odd}}, \end{aligned} \quad r = 0, \dots, m-1.$$

So one FFT stage is a collection of independent **butterflies**

$$(E_r, O_r) \mapsto (E_r + \omega_M^r O_r, E_r - \omega_M^r O_r).$$

This is why FFT maps well to parallel hardware:

- within each stage, many butterflies run independently,
- only $\log_2 M$ stages are needed,
- the pointwise multiply $\hat{K}_r \hat{u}_r$ is also fully parallel over r .

So for S4 training, the whole convolution path is parallel across time positions *because* it is converted into parallel work across frequency indices and butterfly pairs.

Why FFT is parallelizable

The FFT is not just asymptotically faster; it is structured as a *butterfly network*. At each stage, many independent pairs are combined:

$$(a, b) \mapsto (a + \omega b, a - \omega b).$$

So each FFT stage can be executed in parallel across many indices. Hence for S4-style LTI layers:

- the kernel K is fixed for the whole sequence,
- the frequency-domain multiplication is fully parallel across coordinates,
- the inverse FFT is again parallel by stages.

This is the computational reason S4 training can be highly parallel even though the original state equation is recurrent.

What bottleneck remains after the convolution trick?

The FFT helps only *after* we know the kernel

$$K_\ell = C\bar{A}^\ell\bar{B}.$$

So the real problem becomes:

How do we choose A so memory is long and stable, and how do we compute the resulting kernel fast?

That is exactly why we need HiPPO first, and then S4.

What HiPPO is trying to do

HiPPO means **High-Order Polynomial Projection Operators**. The idea is:

- At time t , we do not want to store the entire past function.
- Instead, we want a small vector of coefficients that approximates the whole history.
- The coefficients should update *online* as new data arrives.

So HiPPO asks:

Can we maintain the projection of the past onto an orthogonal polynomial basis by an ODE?

If yes, then the coefficient vector itself is a state space model.

HiPPO clarification: yes, it is remembering a real function f

HiPPO starts from a **real-valued signal**

$$f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}.$$

At time t , the past is the whole real function restricted to $[0, t]$. The goal is to compress that entire past into a small state vector. A convenient rescaling is

$$g_t(x) = f(tx), \quad x \in [0, 1].$$

So the state is trying to remember the function g_t , which is just the past of the original real signal f rewritten on a fixed interval. Important interpretation:

- the object being remembered is literally a *function*, not only a list of tokens,
- HiPPO says the hidden state should store polynomial coefficients of that function,
- vector-valued inputs are handled by running this idea channel-wise or with multi-channel variants.

So yes: the statement “HiPPO remembers a function f ” is exactly right.

The exact optimization problem solved by HiPPO

Fix the orthonormal basis $p_0, \dots, p_{N-1}$ on $[0, 1]$. At time t , HiPPO wants the best $L^2([0, 1])$ approximation of g_t in their span:

$$\min_{\tilde{g} \in \text{span}\{p_0, \dots, p_{N-1}\}} \int_0^1 |g_t(x) - \tilde{g}(x)|^2 dx.$$

Write

$$\tilde{g}(x) = \sum_{n=0}^{N-1} c_n(t) p_n(x).$$

Because the basis is orthonormal, the optimal coefficients are exactly the projection coefficients

$$c_n(t) = \int_0^1 g_t(x) p_n(x) dx.$$

So the state vector

$$c(t) = (c_0(t), \dots, c_{N-1}(t))^{\top}$$

is the best degree- $(N - 1)$ polynomial summary of the past signal at time t . This is the mathematical meaning of “memory” in HiPPO.

Step 1: represent the past as a function on a fixed interval

For the scaled-Legendre construction, we look at the entire past interval $[0, t]$. Rescale time so the past always lives on $[0, 1]$:

$$g_t(x) := f(tx), \quad x \in [0, 1].$$

Interpretation:

- $x = 1$ corresponds to the newest time t ,
- $x = 0$ corresponds to the beginning of the history,
- as t changes, the whole past is re-embedded into the same interval $[0, 1]$.

Now the question becomes: how do we approximate the function g_t on $[0, 1]$?

Step 2: choose an orthogonal basis - Legendre polynomials

On $[-1, 1]$, the Legendre polynomials are the classical orthogonal polynomials

$$P_n(z) = \frac{1}{2^n n!} \frac{d^n}{dz^n} (z^2 - 1)^n.$$

Important properties:

- **Orthogonality:**

$$\int_{-1}^1 P_n(z) P_m(z) dz = \frac{2}{2n+1} \delta_{nm}.$$

- **Recurrence:**

$$(n+1)P_{n+1}(z) = (2n+1)zP_n(z) - nP_{n-1}(z).$$

- **Endpoint value:**

$$P_n(1) = 1.$$

Orthogonality is the key property: it lets coefficients act as a stable, nonredundant summary of the past.

Step 3: shift Legendre polynomials to $[0, 1]$

We need a basis on $[0, 1]$, not on $[-1, 1]$. Map

$$z = 2x - 1.$$

Define shifted orthonormal polynomials

$$p_n(x) := \sqrt{2n+1} P_n(2x-1), \quad x \in [0, 1].$$

Then

$$\int_0^1 p_n(x) p_m(x) dx = \delta_{nm}.$$

So $\{p_0, p_1, \dots, p_{N-1}\}$ is an orthonormal basis for the first N polynomial modes on $[0, 1]$. This is why Legendre polynomials appear in HiPPO: they give an orthogonal polynomial coordinate system for the past.

Step 4: define the coefficient state

Now project the rescaled history g_t onto this basis:

$$c_n(t) := \int_0^1 g_t(x) p_n(x) dx = \int_0^1 f(tx) p_n(x) dx.$$

So the state vector is

$$c(t) = (c_0(t), c_1(t), \dots, c_{N-1}(t))^T.$$

This means

$$g_t(x) \approx \sum_{n=0}^{N-1} c_n(t) p_n(x).$$

Interpretation:

- $c_0(t)$ stores a coarse average mode,
- higher $c_n(t)$ store finer polynomial detail,
- the whole vector $c(t)$ is an N -dimensional summary of the function history.

Step 5: differentiate the coefficients carefully

We want an ODE for $c_n(t)$. Start from

$$c_n(t) = \int_0^1 f(tx) p_n(x) dx.$$

Differentiate with respect to t :

$$\frac{d}{dt} c_n(t) = \int_0^1 x f'(tx) p_n(x) dx.$$

Now use

$$\frac{d}{dx} f(tx) = t f'(tx) \quad \implies \quad x f'(tx) = \frac{x}{t} \frac{d}{dx} f(tx).$$

Hence

$$\frac{d}{dt} c_n(t) = \frac{1}{t} \int_0^1 x p_n(x) \frac{d}{dx} f(tx) dx.$$

That $1/t$ factor is important: the exact scaled-Legendre HiPPO dynamics are *time-varying*.

Step 6: integrate by parts

Apply integration by parts with

$$u(x) = xp_n(x), \quad dv = \frac{d}{dx} f(tx) dx.$$

Then

$$\dot{c}_n(t) = \frac{1}{t} \left[xp_n(x) f(tx) \right]_{x=0}^{x=1} - \frac{1}{t} \int_0^1 \frac{d}{dx} (xp_n(x)) f(tx) dx.$$

Since $x = 0$ kills the lower boundary term and $p_n(1) = \sqrt{2n+1}$,

$$\dot{c}_n(t) = \frac{1}{t} \sqrt{2n+1} f(t) - \frac{1}{t} \int_0^1 \frac{d}{dx} (xp_n(x)) f(tx) dx.$$

So the new sample $f(t)$ appears explicitly as a boundary term.

Step 7: expand the derivative back in the Legendre basis

Because $\{p_k\}$ is a basis, we can expand

$$\frac{d}{dx}(xp_n(x)) = \sum_{k=0}^n S_{nk} p_k(x)$$

for some lower-triangular coefficients S_{nk} . Then

$$\int_0^1 \frac{d}{dx}(xp_n(x)) f(tx) dx = \sum_{k=0}^n S_{nk} \int_0^1 f(tx) p_k(x) dx = \sum_{k=0}^n S_{nk} c_k(t).$$

Therefore

$$\dot{c}_n(t) = -\frac{1}{t} \sum_{k=0}^n S_{nk} c_k(t) + \frac{1}{t} \sqrt{2n+1} f(t).$$

Collecting all coordinates,

$$\dot{c}(t) = -\frac{1}{t} A_{\text{LegS}} c(t) + \frac{1}{t} B_{\text{LegS}} f(t).$$

This is the exact HiPPO-LegS state equation.

Step 8: the explicit HiPPO-LegS matrix

For the scaled Legendre basis, the matrix and input vector are

$$(A_{\text{LegS}})_{nk} = \begin{cases} \sqrt{(2n+1)(2k+1)}, & n > k, \\ n+1, & n = k, \\ 0, & n < k, \end{cases} \quad (B_{\text{LegS}})_n = \sqrt{2n+1}.$$

So the ODE is

$$\dot{c}(t) = -\frac{1}{t}A_{\text{LegS}}c(t) + \frac{1}{t}B_{\text{LegS}}f(t).$$

Equivalent sign conventions sometimes absorb the minus sign into A and write a lower-triangular negative matrix. The content is the same. Important structural facts:

- lower triangular: coefficient n only depends on modes $0, \dots, n$,
- diagonal grows like $n+1$: higher modes decay faster,
- $1/t$ makes the memory scale with elapsed time.

Why Legendre is a good basis for memory

Legendre polynomials are useful here because:

- ① **Orthogonality** prevents redundant coefficients.
- ② **Polynomial hierarchy** gives a clean low-to-high frequency ordering.
- ③ **Explicit formulas** make the state-update matrix computable in closed form.
- ④ **Projection meaning** gives the state an interpretable role: it stores the best N -mode approximation of the past.

So HiPPO is not "just a clever initialization". It says exactly what the state should represent.

What HiPPO gives us, and what it does not

HiPPO gives a principled long-memory operator:

$$\dot{c}(t) = -\frac{1}{t}A_{\text{LegS}}c(t) + \frac{1}{t}B_{\text{LegS}}f(t).$$

But it does *not yet* solve the full deep-learning problem. We still need:

- a stable discretization for sequences,
- a trainable parameterization,
- a fast way to compute the convolution kernel.

That is exactly the job of S4.

S4 starting point

S4 starts from a continuous-time SSM

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t).$$

Conceptually, HiPPO tells us what a good long-memory A should look like. S4 then asks:

How can we turn such a continuous-time long-memory system into a trainable, fast sequence layer?

The answer has three pieces:

- ① stable discretization,
- ② structured parameterization of A ,
- ③ kernel computation via resolvents, Cauchy kernels, and FFT.

What HiPPO contributes to S4, exactly

HiPPO is *not* the whole S4 model. Its role is more specific:

- ① HiPPO provides a principled long-memory continuous-time operator ($A_{\text{HiPPO}}, B_{\text{HiPPO}}$).
- ② S4 starts from this operator as an **initialization / structural template** for memory.
- ③ S4 then applies the low-rank correction and diagonalization trick to obtain the DPLR form

$$A = \Lambda - pq^*.$$

- ④ After that, S4 **learns** the model parameters end-to-end.

So the role of HiPPO inside S4 is:

give a mathematically meaningful long-memory starting dynamics.

Then S4 adds the computational machinery: discretization, DPLR parameterization, Cauchy-kernel evaluation, and FFT training.

S4 step 1: stable discretization by the bilinear transform

For sequence modeling we need a discrete recurrence

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k.$$

S4 uses the bilinear (Tustin) discretization

$$\bar{A} = \left(I - \frac{\Delta}{2}A\right)^{-1} \left(I + \frac{\Delta}{2}A\right), \quad \bar{B} = \left(I - \frac{\Delta}{2}A\right)^{-1} \Delta B.$$

Why this one?

- it maps stable continuous-time eigenvalues to stable discrete-time eigenvalues,
- it exposes a very useful resolvent identity,
- Δ is learnable, so the model can choose its own timescale.

After this, the kernel is still

$$K_\ell = C\bar{A}^\ell\bar{B}.$$

S4 step 2: do not compute matrix powers directly

Naively we would need

$$K_0, K_1, \dots, K_{L-1}, \quad K_\ell = C\bar{A}^\ell\bar{B}.$$

That means computing many powers of $\bar{A}$, which is too expensive for large L . So S4 changes viewpoint:

Instead of computing the kernel in time directly, compute its generating function in frequency space.

This is the key computational trick.

S4 step 3: introduce the generating function

Define

$$\hat{\mathcal{K}}_L(z) := \sum_{\ell=0}^{L-1} K_{\ell} z^{\ell} = \sum_{\ell=0}^{L-1} C \bar{A}^{\ell} \bar{B} z^{\ell}.$$

This is a truncated matrix-valued geometric series. For any z such that $z^L = 1$,

$$\sum_{\ell=0}^{L-1} (\bar{A}z)^{\ell} = (I - (\bar{A}z)^L)(I - \bar{A}z)^{-1}.$$

Hence

$$\hat{\mathcal{K}}_L(z) = C(I - \bar{A}^L)(I - \bar{A}z)^{-1}\bar{B}.$$

Define

$$\tilde{C} := C(I - \bar{A}^L).$$

Then

$$\boxed{\hat{\mathcal{K}}_L(z) = \tilde{C}(I - \bar{A}z)^{-1}\bar{B}.}$$

Now the hard object is one inverse, not many powers.

S4 step 4: rewrite the discrete resolvent in terms of A

Substitute the bilinear formula for $\bar{A}$:

$$I - \bar{A}z = I - z \left(I - \frac{\Delta}{2}A \right)^{-1} \left(I + \frac{\Delta}{2}A \right).$$

Factor out $\left(I - \frac{\Delta}{2}A \right)^{-1}$:

$$I - \bar{A}z = \left(I - \frac{\Delta}{2}A \right)^{-1} \left[(1 - z)I - \frac{\Delta}{2}(1 + z)A \right].$$

Therefore

$$(I - \bar{A}z)^{-1} \bar{B} = \frac{2}{1 + z} \left(\frac{2}{\Delta} \frac{1 - z}{1 + z} I - A \right)^{-1} B.$$

Define

$$g(z) := \frac{2}{\Delta} \frac{1 - z}{1 + z}, \quad c(z) := \frac{2}{1 + z}.$$

Then

$$\boxed{\hat{\mathcal{K}}_L(z) = c(z) \tilde{C} (g(z)I - A)^{-1} B.}$$

S4 step 5: parameterize A as diagonal plus low rank

This is the central S4 parameterization. Instead of learning an arbitrary dense matrix A , S4 works in a basis where

$$A = \Lambda - pq^*,$$

with

$$\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N) \in \mathbb{C}^{N \times N}, \quad p, q \in \mathbb{C}^N.$$

This is called **DPLR** = diagonal plus low rank. Why this form?

- diagonal part is easy to invert and exponentiate,
- low-rank correction keeps enough structure to approximate the HiPPO operator,
- Woodbury identity turns the inverse of diagonal-plus-low-rank into a small correction of a diagonal inverse.

This is the point where S4 turns HiPPO into something computationally tractable.

What exactly is learned in S4?

At the SSM level, the learned parameters are the structured pieces

$$\Lambda, p, q, B, C, \Delta,$$

plus the surrounding neural-network mixing/projection weights. Interpretation:

- Λ sets the main memory modes / poles,
- p, q provide the low-rank correction,
- B controls how input excites the modes,
- C controls how the modes are read out,
- Δ controls the discretization timescale.

In practice the system is often initialized from a HiPPO-inspired operator and then trained end-to-end.

S4 step 6: the diagonal case already gives a Cauchy kernel

If A were purely diagonal,

$$A = \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N),$$

then

$$(g(z)I - \Lambda)^{-1} = \text{diag}\left(\frac{1}{g(z) - \lambda_1}, \dots, \frac{1}{g(z) - \lambda_N}\right).$$

So

$$\hat{\mathcal{K}}_L(z) = c(z) \sum_{i=1}^N \frac{\tilde{C}_i B_i}{g(z) - \lambda_i}.$$

This is exactly a Cauchy-type sum:

$$k(\omega) = \sum_{i=1}^N \frac{v_i}{\omega - \lambda_i}.$$

So the diagonal case is already cheap: it reduces to evaluating a rational function at many frequencies.

S4 step 7: now use the real DPLR form and Woodbury

With the actual S4 parameterization

$$A = \Lambda - pq^*,$$

we get

$$g(z)I - A = D(z) + pq^*, \quad D(z) := g(z)I - \Lambda.$$

Apply Woodbury:

$$(D + pq^*)^{-1} = D^{-1} - D^{-1}p(1 + q^*D^{-1}p)^{-1}q^*D^{-1}.$$

Substitute into the kernel formula:

$$\hat{\mathcal{K}}_L(z) = c(z) \left[\tilde{C}D^{-1}B - \tilde{C}D^{-1}p(1 + q^*D^{-1}p)^{-1}q^*D^{-1}B \right].$$

This is the exact step that turns the hard inverse into diagonal inverses plus a scalar low-rank correction.

S4 step 8: reduce to four Cauchy bilinear forms

Define

$$k_z(a, b) := \sum_{i=1}^N \frac{a_i b_i}{g(z) - \lambda_i}.$$

Then the previous expression becomes

$$\hat{\mathcal{K}}_L(z) = c(z) \left[k_z(\tilde{C}, B) - k_z(\tilde{C}, p)(1 + k_z(q^*, p))^{-1} k_z(q^*, B) \right].$$

So one frequency evaluation needs exactly four Cauchy-type quantities:

$$k_z(\tilde{C}, B), \quad k_z(\tilde{C}, p), \quad k_z(q^*, p), \quad k_z(q^*, B).$$

This is the famous “S4 reduces to four Cauchy kernels” statement.

S4 step 9: use the frequency response to apply the layer

Now evaluate the generating function at FFT frequencies

$$\omega_j = e^{-2\pi ij/M}, \quad j = 0, 1, \dots, M-1,$$

after zero-padding to length M . Define

$$\hat{K}_j := \hat{K}_M(\omega_j).$$

These are the Fourier coefficients of the padded convolution kernel. To apply the layer to an input sequence u , do *not* inverse-transform the kernel first unless you explicitly want to materialize it. Instead:

- 1 compute $\hat{K}_j$ for all frequencies from the DPLR formula,
- 2 compute $\hat{u} = \text{FFT}(u^{\text{pad}})$,
- 3 multiply pointwise

$$\hat{y}_j = \hat{K}_j \hat{u}_j,$$

- 4 return to time by

$$y^{\text{pad}} = \text{IFFT}(\hat{y}),$$

- 5 keep the first L output positions.

Only if you explicitly want the kernel itself do you compute

S4 step 10: exact FFT pipeline in practice

S4 does *not* first build $K_\ell = C\bar{A}^\ell\bar{B}$ by explicit powers. Instead it builds the generating function

$$\hat{\mathcal{K}}_M(z) = \sum_{\ell=0}^{M-1} K_\ell z^\ell$$

and evaluates it at the FFT roots of unity

$$z_r = \omega_M^r, \quad r = 0, \dots, M-1.$$

Those evaluations are exactly the Fourier samples of the padded kernel:

$$\hat{K}_r = \hat{\mathcal{K}}_M(\omega_M^r).$$

Then the **actual forward pass** is

$$\hat{y}_r = \hat{K}_r \hat{u}_r \quad \text{for all } r,$$

followed by one inverse FFT. So the computational pipeline is

$$(\Lambda, p, q, B, C, \Delta) \rightarrow \hat{K}_r \rightarrow \hat{y}_r = \hat{K}_r \hat{u}_r \rightarrow y.$$

Why S4 is fast

S4 is fast for three linked reasons:

- ① **Structured parameterization:** $A = \Lambda - pq^*$ makes the resolvent tractable.
- ② **Cauchy reduction:** each frequency evaluation is reduced to a few structured sums.
- ③ **FFT application:** once the frequency response is known, the input is transformed, multiplied pointwise in frequency, and inverse-transformed; all frequency coordinates are processed in parallel.

In one sentence:

S4 is a long convolution whose filter is generated by a structured state space model.

Why we need something beyond S4

S4 is still *linear time-invariant*:

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k, \quad y_k = Cx_k.$$

That means the same dynamics are used for every token. For language, this is often too rigid:

- some tokens should be ignored,
- some should be stored strongly,
- some should change the memory timescale immediately.

Mamba keeps the SSM idea but makes the effective update depend on the current input.

Mamba detail: token-dependent parameters before the scan

Mamba keeps the SSM idea but makes some parameters depend on the current input:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t + D x_t.$$

A common discretized diagonal view is

$$\bar{A}_t = \exp(\Delta_t A), \quad \bar{B}_t = (\Delta_t A)^{-1} (\exp(\Delta_t A) - I) \Delta_t B_t.$$

Because Δ_t , B_t , and C_t depend on x_t , the dynamics are **time-varying**. This is the new modeling power:

- some tokens can be forgotten quickly,
- some tokens can be written strongly,
- some tokens can be read out strongly later.

But now one fixed convolution kernel no longer exists.

Mamba step 1: selective parameterization

Mamba starts from a continuous-time SSM viewpoint, but makes several parameters input-dependent. A clean schematic form is:

$$h'(t) = Ah(t) + B(x_t)x(t), \quad y(t) = C(x_t)h(t) + Dx(t),$$

and after discretization, the per-token parameters are

$$\Delta_t, \quad B_t, \quad C_t.$$

With zero-order hold discretization,

$$\bar{A}_t = e^{\Delta_t A}, \quad \bar{B}_t = \left(\int_0^{\Delta_t} e^{\tau A} d\tau \right) B_t.$$

Hence the selective recurrence is

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t + D x_t.$$

The important change is: $\bar{A}_t, \bar{B}_t, C_t$ **can vary with the token.**

Mamba step 2: why this breaks the S4 convolution trick

Unroll the recurrence:

$$h_t = \bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_1 h_0 + \sum_{j=1}^t \bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_{j+1} \bar{B}_j x_j.$$

Compare with S4:

$$h_t = \sum_{j=1}^t \bar{A}^{t-j} \bar{B} x_j.$$

In S4, the coefficient on x_j depends only on the lag $t - j$. Therefore the operator is Toeplitz and becomes a convolution. In Mamba, the coefficient depends on the entire interval

$$\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_{j+1},$$

so it depends on *where* you are, not just on the lag. Therefore there is no single global convolution kernel and no FFT shortcut.

Mamba step 3: specialize to diagonal or elementwise dynamics

Mamba uses a particularly efficient case where the state dynamics are diagonal / elementwise. Then each state coordinate evolves like

$$h_t = a_t \odot h_{t-1} + b_t,$$

where

$$a_t := e^{\Delta_t A}, \quad b_t := \bar{B}_t x_t.$$

This is still a recurrence, but now each step is an affine map on the state:

$$\Phi_t(h) = a_t \odot h + b_t.$$

Represent the step by the pair

$$s_t = (a_t, b_t).$$

This is the algebraic form that makes the selective scan possible.

Mamba step 4: compose two affine updates

Take two consecutive updates:

$$\Phi_1(h) = a_1 \odot h + b_1, \quad \Phi_2(h) = a_2 \odot h + b_2.$$

Then

$$\begin{aligned}\Phi_2(\Phi_1(h)) &= a_2 \odot (a_1 \odot h + b_1) + b_2 \\ &= (a_2 \odot a_1) \odot h + (a_2 \odot b_1 + b_2).\end{aligned}$$

So the composed step is again affine:

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 \odot a_1, a_2 \odot b_1 + b_2).$$

This closed-form composition law is the new algebraic ingredient in Mamba.

Mamba step 5: prove associativity

Now check associativity explicitly.

$$\begin{aligned}((a_3, b_3) \circ (a_2, b_2)) \circ (a_1, b_1) &= (a_3 \odot a_2 \odot a_1, a_3 \odot a_2 \odot b_1 + a_3 \odot b_2 + b_3), \\(a_3, b_3) \circ ((a_2, b_2) \circ (a_1, b_1)) &= (a_3 \odot a_2 \odot a_1, a_3 \odot a_2 \odot b_1 + a_3 \odot b_2 + b_3).\end{aligned}$$

They match exactly, so

$$(s_3 \circ s_2) \circ s_1 = s_3 \circ (s_2 \circ s_1).$$

That means all prefix products

$$s_t \circ s_{t-1} \circ \cdots \circ s_1$$

can be computed by a **parallel associative scan**.

Mamba step 6: recover the hidden state from the scan

Suppose the scan returns

$$(\alpha_t, \beta_t) = s_t \circ s_{t-1} \circ \cdots \circ s_1.$$

Then the action of the whole prefix on the initial state is

$$h_t = \alpha_t \odot h_0 + \beta_t.$$

So the selective scan algorithm is:

- 1 build local pairs $s_t = (a_t, b_t)$ from the input,
- 2 prefix-scan them with the associative rule

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 \odot a_1, a_2 \odot b_1 + b_2),$$

- 3 recover all hidden states from (α_t, β_t) ,
- 4 apply the readout $y_t = C_t h_t + D x_t$.

So Mamba replaces the S4 FFT with a scan over affine transformations.

Mamba detail: the parallel prefix-scan algorithm

For an associative operator $\circ$, the Blelloch scan has two tree phases. **Upsweep (reduce):**

- combine neighboring pairs,
- then combine blocks of size 4, then 8, and so on,
- build a binary tree of block summaries.

Downsweep (prefix propagation):

- send each block the prefix summary of everything before it,
- combine with its local summary to produce child prefixes,
- continue recursively until every position has its prefix.

Applied to Mamba, the data at each leaf is $s_t = (a_t, b_t)$. The output at leaf t is

$$S_t = s_t \circ s_{t-1} \circ \cdots \circ s_1.$$

The total work is $O(L)$ and the parallel depth is $O(\log L)$.

Mamba detail: full selective scan recipe from input to output

Putting everything together, one selective SSM pass is:

- 1 Input projections produce token-dependent quantities Δ_t, B_t, C_t .
- 2 Discretize the continuous dynamics to obtain per-token recurrence coefficients.
- 3 Form the affine step objects $s_t = (a_t, b_t)$.
- 4 Prefix-scan all s_t with the associative composition law.
- 5 Recover hidden states by

$$h_t = \alpha_t \odot h_0 + \beta_t.$$

- 6 Read out

$$y_t = C_t h_t + D x_t.$$

So the exact computational contrast is:

S4: DPLR $\rightarrow$ Cauchy kernel $\rightarrow$ FFT convolution,

Mamba: selective discretization $\rightarrow$ affine pairs $\rightarrow$ parallel scan.

Mamba step 7: why this is still fast in practice

The scan is efficient because:

- the recurrence is still linear in the sequence length,
- associativity enables parallel prefix computation,
- modern implementations fuse discretization, scan, and readout to reduce memory traffic,
- chunking and recomputation keep memory usage manageable for long sequences.

So the computational story is:

S4 gets speed from convolution + FFT. Mamba gets speed from affine associativity + scan.

What is genuinely new in Mamba?

The key new idea is not just “another SSM”. It is this exact shift:

fixed LTI dynamics $\longrightarrow$ input-dependent selective dynamics.

That causes two consequences:

- ① **Modeling gain:** the layer can choose what to keep and what to forget based on content.
- ② **Algorithmic change:** convolution is lost, so scan replaces FFT.

This is why Mamba is best understood as a *selective state space model*, not as a small variation of S4.

S4 vs. Mamba in one table

Aspect	S4	Mamba
Dynamics	fixed LTI	token-dependent selective
Fast primitive	FFT convolution	associative scan
Kernel	one global K	no single global kernel
Main math trick	DPLR + Woodbury + Cauchy	affine composition + scan
Best intuition	learned long filter	learned selective memory machine

Take-home arc of the whole lecture

- ① **Kalman**: hidden state is the right abstraction for memory.
- ② **Convolution theorem**: LTI SSMs are long convolutions.
- ③ **FFT**: fixed kernels make training parallelizable.
- ④ **HiPPO**: choose the state so it stores online polynomial projections of the past.
- ⑤ **S4**: parameterize the HiPPO-like dynamics as DPLR and compute the kernel by Cauchy sums plus FFT.
- ⑥ **Mamba**: make the dynamics selective and recover efficiency by associative scan.

That is the clean mathematical path from classical filtering to modern sequence models.